

MINI PROJECT

Subscription Manager

เว็บจัดการบริการสมัครสมาชิก Full-stack ด้วย REST API

หัวข้อ	รายละเอียด
หัวข้อ	เว็บจัดการบริการรายเดือน/รายปี และสรุปค่าใช้จ่าย
เทคโนโลยี	Node.js, Express.js, HTML, CSS, JavaScript (fetch)
Resource	subscriptions
ชื่อ	ณฤดล บัวนวน
รายวิชา	618498 Selected Topics in Electronics and Computer Engineering I
รหัสนักศึกษา	660910313

รายงานฉบับนี้อธิบายแนวคิด การออกแบบ REST API การทำงานของหน้าเว็บ และผลการทดสอบของโปรเจกต์

1. แนวคิดและวัตถุประสงค์

Subscription Manager เป็นเว็บแอปให้ผู้ใช้บันทึกบริการแบบสมัครสมาชิก เช่น Netflix, Spotify, iCloud ทั้งรายเดือนและรายปี ระบบสรุปค่าใช้จ่ายต่อเดือนและต่อปี แสดงสัดส่วนตามหมวด และเตือนเมื่อใกล้ถึงวันชำระ ผู้ใช้ค้นหา กรอง เรียงลำดับ แก้ไข และลบรายการได้โดยไม่ต้องโหลดหน้าใหม่

วัตถุประสงค์

- ฝึกพัฒนาเว็บ Full-stack ที่มี frontend และ backend แยกส่วนชัดเจน
- ฝึกออกแบบ CRUD REST API และจัดการสถานะ HTTP เช่น 201, 204, 400 และ 404
- ฝึกใช้ fetch() เพื่อรับส่งข้อมูลโดยไม่ต้องโหลดหน้าเว็บใหม่
- ออกแบบหน้าเว็บด้วย CSS ให้ใช้งานง่าย รองรับมือถือและพิมพ์

หมวดบริการ

หมวด	ตัวอย่างบริการ
บันเทิง (entertainment)	Netflix, YouTube Premium
เพลง (music)	Spotify, Apple Music
คลาวด์ (cloud)	iCloud, Google One
การเรียนรู้ (education)	Duolingo, Coursera
อื่น ๆ (other)	บริการนอกเหนือจากหมวดข้างต้น

โครงสร้างระบบ

ผู้ใช้	→	HTML + CSS + JavaScript แสดงผลและเรียก fetch()	→	Express REST API ตรวจสอบข้อมูลและจัดการรายการ
--------	---	--	---	---

2. การออกแบบ REST API

Resource หลักใช้ชื่อ subscriptions เซิร์ฟเวอร์ตรวจสอบข้อมูลก่อนบันทึก และตอบสถานะ HTTP ให้ตรงกับผลลัพธ์

Method	Endpoint	การทำงาน	สถานะ
GET	/api/subscriptions	อ่านรายการทั้งหมด และกรองด้วย query string	200
GET	/api/subscriptions/:id	อ่านรายการตาม ID	200 / 404
POST	/api/subscriptions	เพิ่มรายการใหม่	201 / 400
PATCH	/api/subscriptions/:id	แก้ไขบางฟิลด์ (เช่น ราคา สถานะ)	200 / 400 / 404
DELETE	/api/subscriptions/:id	ลบรายการ	204 / 404

Query string ที่รองรับ

- billing=yearly กรองตามรอบชำระ (monthly / yearly)
- category=music กรองตามหมวด
- active=true กรองตามสถานะการใช้งาน
- q=net ค้นหาจากชื่อบริการ
- sort=price&order=desc เรียงตาม price, nextPayment หรือ name

ตัวอย่าง Request Body

```
{
  "name": "Spotify",
  "price": 129,
  "billing": "monthly",
  "category": "music",
  "nextPayment": "2026-10-15"
}
```

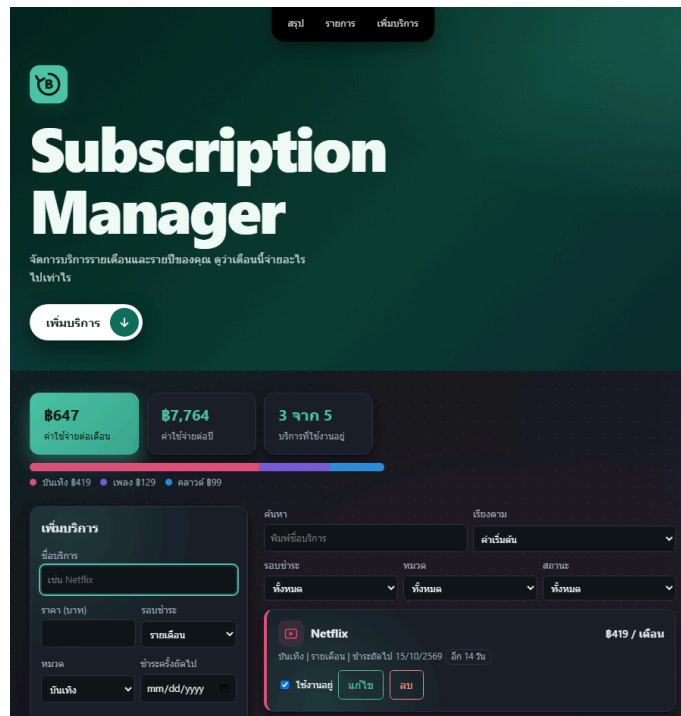
ตัวอย่างการใช้ fetch()

```
fetch('/api/subscriptions', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(data)
})
```

ข้อมูลเก็บในหน่วยความจำของเซิร์ฟเวอร์ / ไฟล์ JSON / ฐานข้อมูล (เลือกอย่างใดอย่างหนึ่ง) การตรวจสอบข้อมูล: POST ต้องมี name, price และ billing หากไม่ครบ เซิร์ฟเวอร์ตอบ 400 พร้อม {"error": "ต้องระบุ name, price และ billing"}

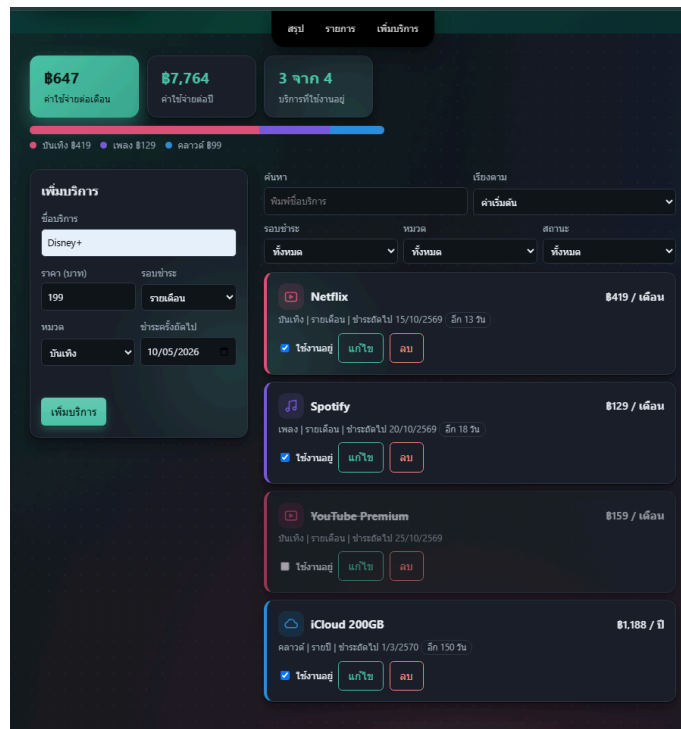
3. หน้าจอและการทำงาน

หน้าเว็บแบ่งเป็นส่วนหัว พร้อมแถบเมนู ส่วนสรุปค่าใช้จ่ายและสัดส่วนตามหมวด พร้อมเพิ่ม/แก้ไขทางซ้าย และรายการบริการทางขวา ทุกการกระทำเรียก fetch() ไปยัง API ของตัวเองแล้วอัปเดตหน้าจอทันที

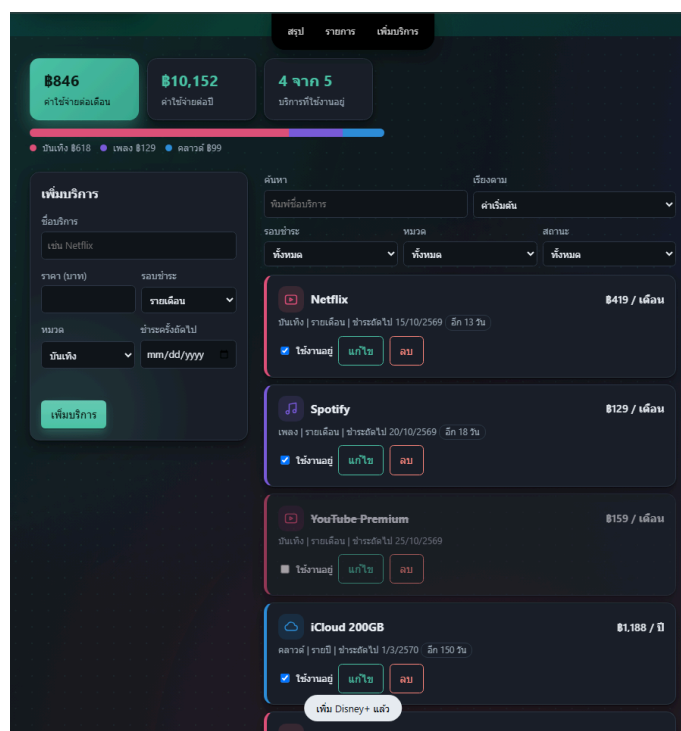


ภาพที่ 1 หน้าหลักของ Subscription Manager

การเพิ่มบริการ: กรอกชื่อ ราคา รอบชำระ หมวด และวันชำระถัดไป แล้วกดเพิ่ม ระบบเรียก POST แล้วแสดงรายการใหม่พร้อมข้อความแจ้งเตือนที่ ยอดสรุปและแถบสัดส่วนอัปเดตตามโดยไม่โหลดหน้าใหม่



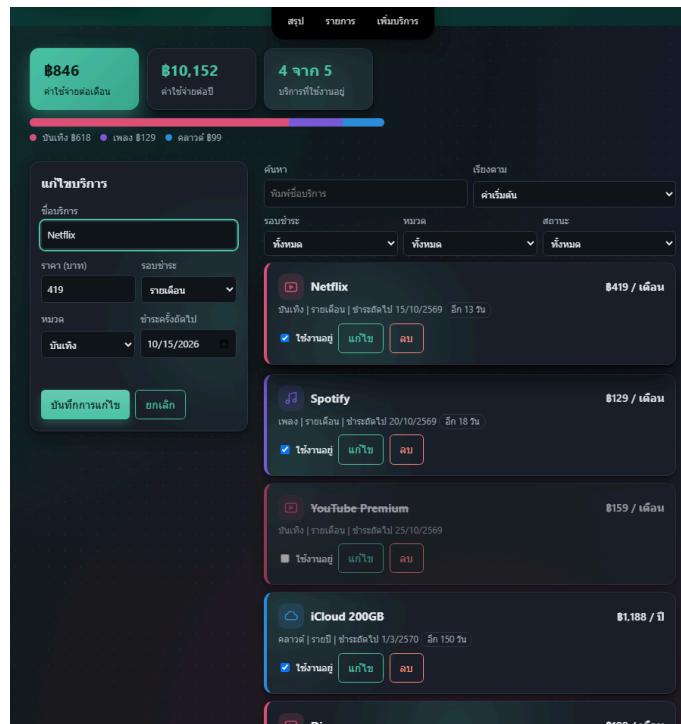
ภาพที่ 2 กรอกฟอร์มก่อนกดเพิ่มบริการ



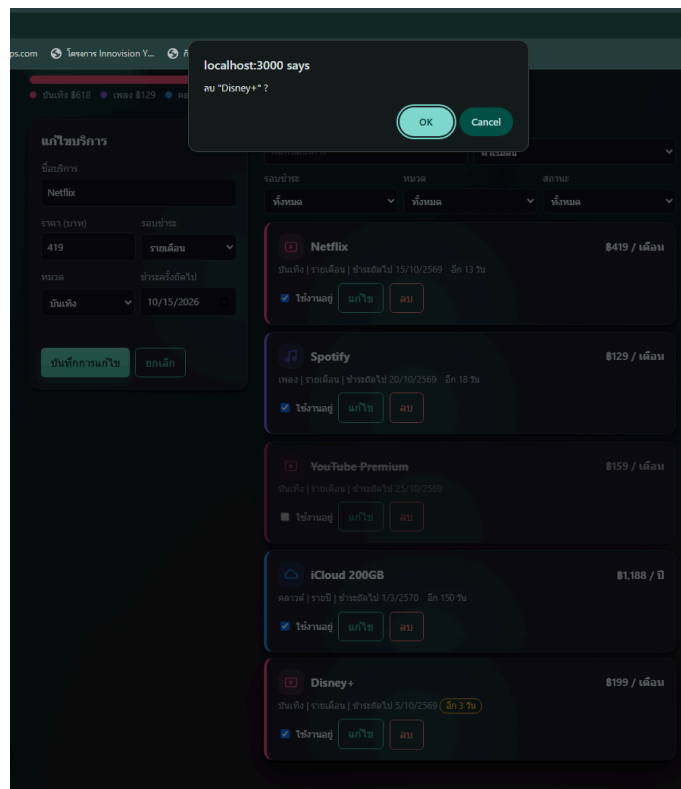
ภาพที่ 3 หลังกดเพิ่ม แสดงข้อความ "เพิ่ม Disney+ แล้ว" และรายการใหม่

การแก้ไขและลบ: ปุ่มแก้ไขนำข้อมูลเดิมขึ้นฟอร์มแล้วเรียก PATCH เมื่อบันทึก ปุ่มลบเรียก DELETE โดยมีกล่องยืนยันก่อนลบ

SUBSCRIPTION MANAGER - MINI PROJECT



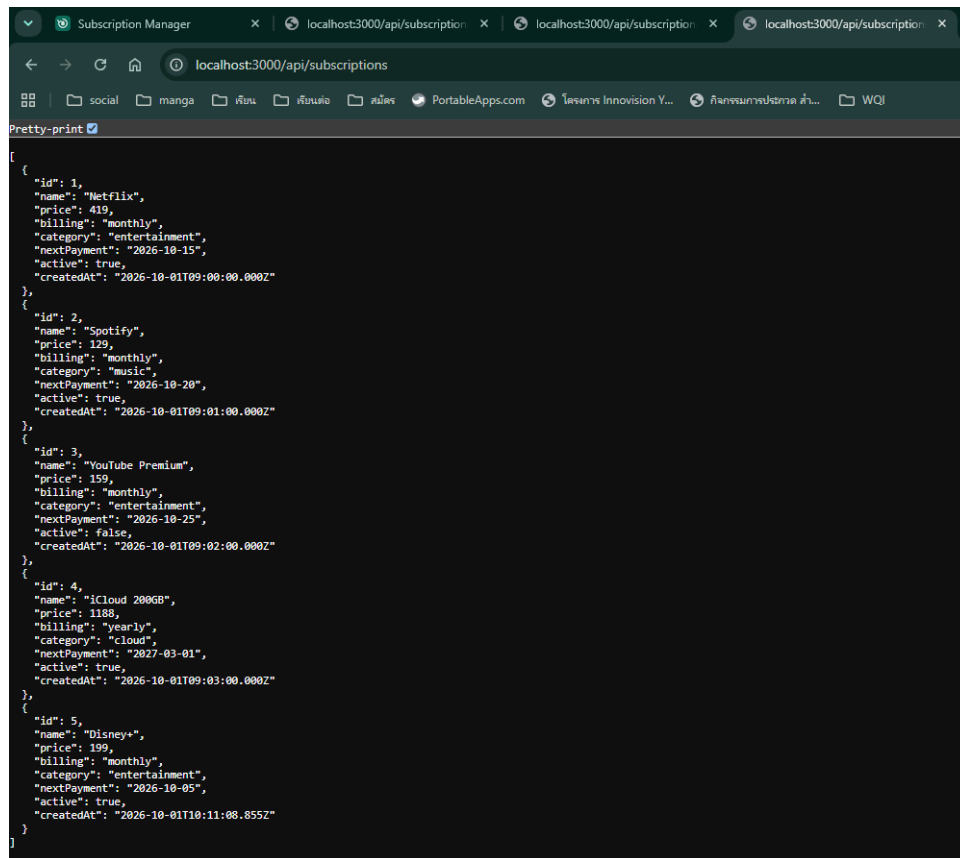
ภาพที่ 4 โหมดแก้ไขบริการ (Netflix)



ภาพที่ 5 กล้องยืนยันก่อนลบรายการ

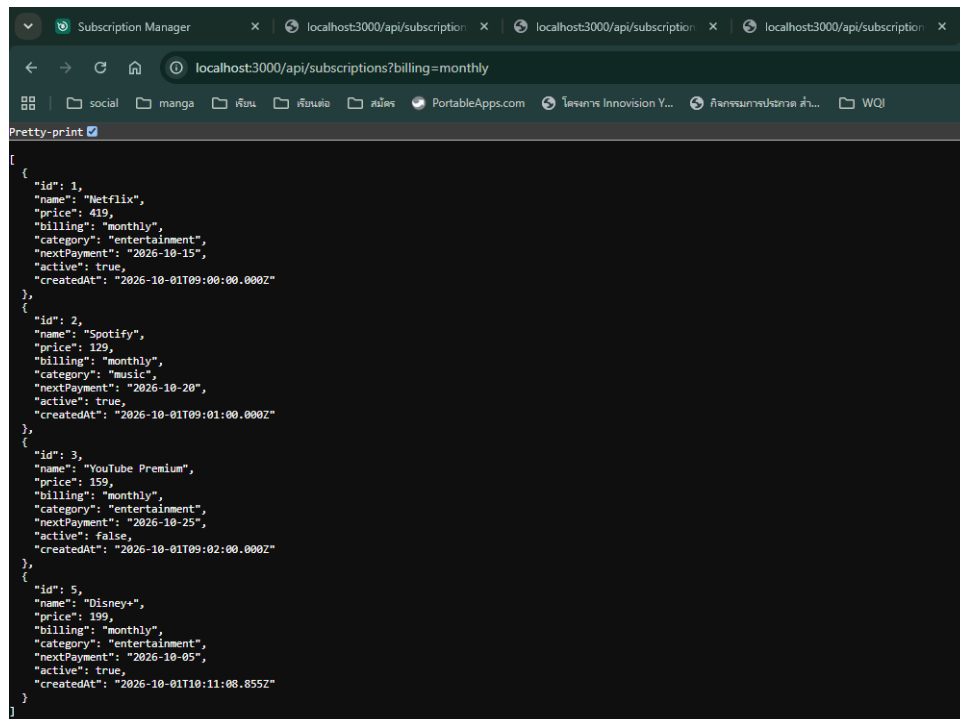
4. หลักการทำงานของ API

ภาพด้านล่างเป็นผลตอบกลับ JSON จากการเรียก API โดยตรงผ่านเบราว์เซอร์



```
[{"id": 1, "name": "Netflix", "price": 419, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-15", "active": true, "createdAt": "2026-10-01T09:00:00.000Z"}, {"id": 2, "name": "Spotify", "price": 129, "billing": "monthly", "category": "music", "nextPayment": "2026-10-20", "active": true, "createdAt": "2026-10-01T09:01:00.000Z"}, {"id": 3, "name": "YouTube Premium", "price": 159, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-25", "active": false, "createdAt": "2026-10-01T09:02:00.000Z"}, {"id": 4, "name": "iCloud 200GB", "price": 1188, "billing": "yearly", "category": "cloud", "nextPayment": "2027-03-01", "active": true, "createdAt": "2026-10-01T09:03:00.000Z"}, {"id": 5, "name": "Disney+", "price": 199, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-05", "active": true, "createdAt": "2026-10-01T10:11:08.855Z"}]
```

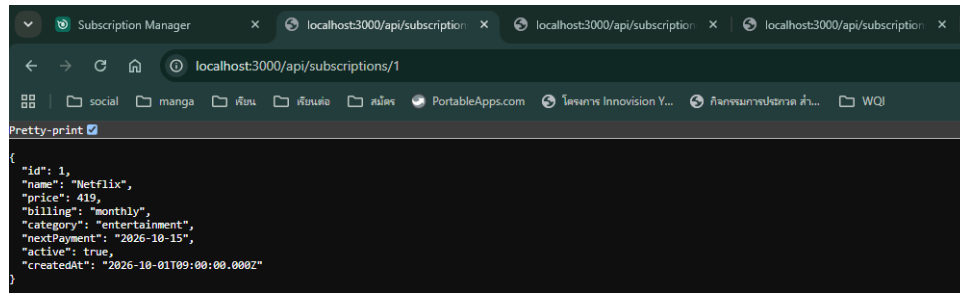
ภาพที่ 6 GET /api/subscriptions ได้รายการทั้งหมด (200)



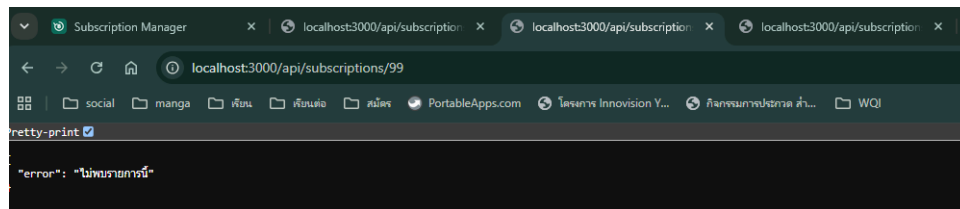
```
[{"id": 1, "name": "Netflix", "price": 419, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-15", "active": true, "createdAt": "2026-10-01T09:00:00.000Z"}, {"id": 2, "name": "Spotify", "price": 129, "billing": "monthly", "category": "music", "nextPayment": "2026-10-20", "active": true, "createdAt": "2026-10-01T09:01:00.000Z"}, {"id": 3, "name": "YouTube Premium", "price": 159, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-25", "active": false, "createdAt": "2026-10-01T09:02:00.000Z"}, {"id": 5, "name": "Disney+", "price": 199, "billing": "monthly", "category": "entertainment", "nextPayment": "2026-10-05", "active": true, "createdAt": "2026-10-01T10:11:08.855Z"}]
```

ภาพที่ 7 GET /api/subscriptions?billing=monthly กรองด้วย query string

SUBSCRIPTION MANAGER - MINI PROJECT

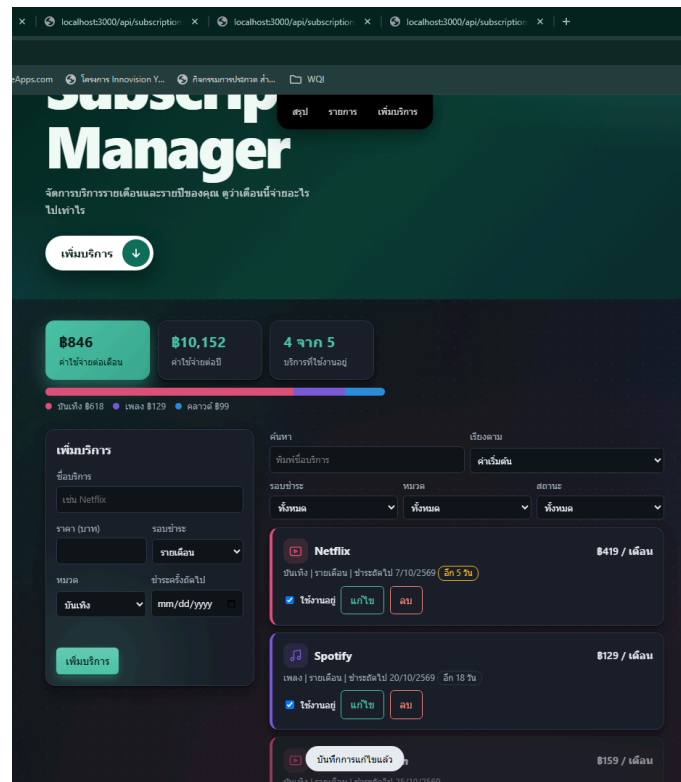


ภาพที่ 8 GET /api/subscriptions/1 ได้รายการเดียวด้วย route parameter (200)

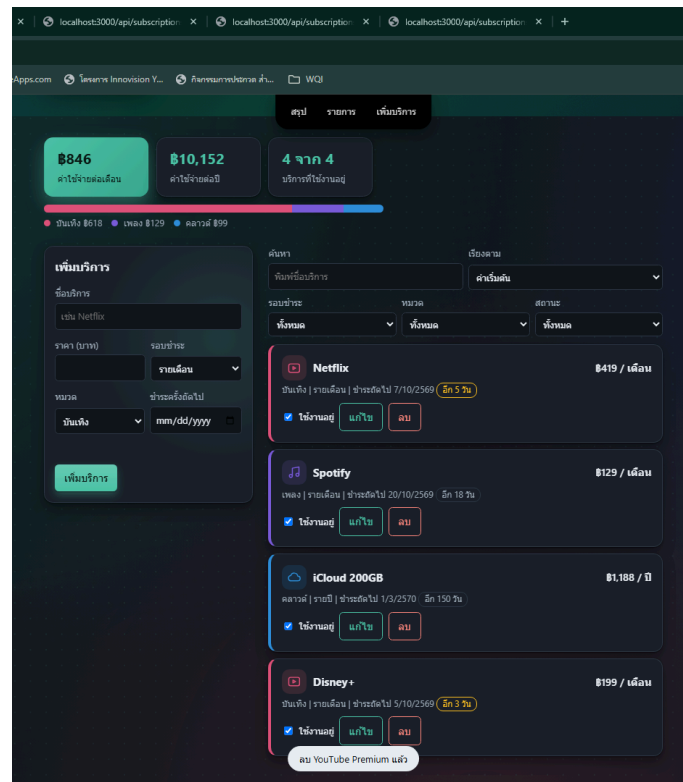


ภาพที่ 9 GET /api/subscriptions/99 ไม่พบรายการ ตอบข้อความ error (404)

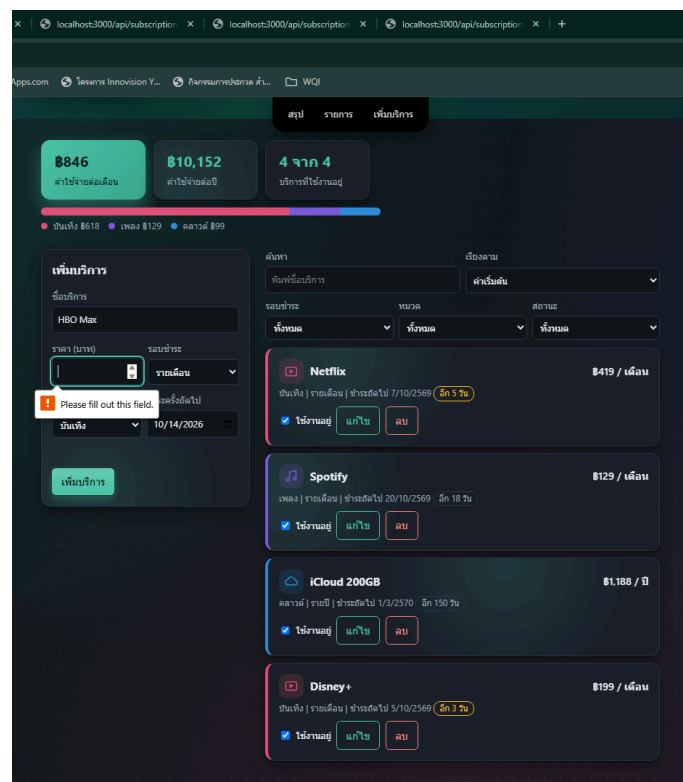
หลักฐานเพิ่มเติม: แก้ไข ลบ และตรวจสอบข้อมูล



ภาพที่ 10 หลังกดบันทึกการแก้ไข (PATCH) แสดงข้อความ "บันทึกการแก้ไขแล้ว" และวันชำระของ Netflix เปลี่ยนเป็น 7/10/2569



ภาพที่ 11 หลังกดลบ (DELETE) YouTube Premium หายจากรายการ และยอดเปลี่ยนเป็น 4 จาก 4



ภาพที่ 12 ฟอर्मตรวจสอบข้อมูลก่อนส่ง เมื่อไม่กรอกราคาจะแจ้งให้กรอกข้อมูล

```

▶ 7 Unchecked runtime.lastError: Could not establish connection. Receiving
end does not exist.

> fetch('/api/subscriptions', {method:'POST', headers:{'Content-
Type':'application/json'}, body:JSON.stringify({name:'HBO Max'})}).then(r
=> r.json().then(d => console.log(r.status, d)))
< ▶ Promise {<pending>}

✖ ▶ POST http://localhost:3000/api/subscriptions 400 (Bad Request) VM220:1
400 ▶ {error: 'ต้องระบุ name, price และ billing'} VM220:1

```

ภาพที่ 13 ส่ง POST ข้อมูลไม่ครบด้วย fetch() ใน Console เซิร์ฟเวอร์ตอบ 400 (Bad Request) พร้อมข้อความ error

```

> fetch('/api/subscriptions/99', {method:'PATCH', headers:{'Content-
Type':'application/json'}, body:JSON.stringify({price:1})}).then(r =>
r.json().then(d => console.log(r.status, d)))
< ▶ Promise {<pending>}

✖ ▶ PATCH http://localhost:3000/api/subscriptions/99 404 (Not Found) VM224:1
404 ▶ {error: 'ไม่พบรายการนี้'} VM224:1

```

ภาพที่ 14 ส่ง PATCH ไปยัง ID ที่ไม่มี (/99) เซิร์ฟเวอร์ตอบ 404 (Not Found) พร้อมข้อความ error

ผลการทดสอบ

รายการ	ผล
GET รายการทั้งหมด และ query string (ภาพที่ 6, 7)	ผ่าน
GET /:id ที่พบ (ภาพที่ 8)	ผ่าน
GET /:id ที่ไม่พบ ตอบ 404 (ภาพที่ 9)	ผ่าน
POST ข้อมูลถูกต้อง ตอบ 201 (ภาพที่ 3)	ผ่าน
POST ข้อมูลไม่ครบ ตอบ 400 (ภาพที่ 13)	ผ่าน
PATCH แก้ไขรายการสำเร็จ (ภาพที่ 10)	ผ่าน
PATCH ID ที่ไม่พบ ตอบ 404 (ภาพที่ 14)	ผ่าน
DELETE ลบรายการสำเร็จ (ภาพที่ 11)	ผ่าน
ตรวจสอบข้อมูลฝั่งฟอร์ม (ภาพที่ 12)	ผ่าน

5. วิธีติดตั้งและรัน

1. ติดตั้ง Node.js แล้วเปิด Terminal ที่โฟลเดอร์โปรเจกต์

```
npm install  
npm run dev
```

2. เปิดหน้าเว็บที่ <http://localhost:3000>

โครงสร้างไฟล์สำคัญ

ไฟล์	หน้าที่
server/	Express server และ CRUD REST API
public/index.html	โครงหน้าเว็บ
public/style.css	การตกแต่งและธีมสว่าง/มืด
public/app.js	เรียก fetch() และแสดงผล
README.md	คู่มือติดตั้ง วิธีรัน และรายการ endpoint

สรุป

โปรเจกต์นี้ทำตามเงื่อนไข Mini Project ได้แก่ มี frontend และ backend แยกชัดเจน ใช้ REST API ของตนเอง รองรับ CRUD ครบ มี query string อย่างน้อยหนึ่งพารามิเตอร์ มีการตรวจสอบข้อมูล มีสถานะ 400 และ 404 และหน้าเว็บเรียก API ด้วย fetch() โดยไม่โหลดหน้าใหม่

Repository สำหรับส่งงาน: https://github.com/1nicotine/Subscription-Manager_MiniProject